



Guía de Implementación Técnica y Roadmap de Desarrollo End-to-End

TECH SPECIFICATION & DEVELOPER PLAYBOOK

Este documento constituye el plan de ejecución paso a paso para el equipo de ingeniería de software. Detalla la construcción del proyecto **SyncReact** desde el setup del entorno hasta la lógica de sincronización de baja latencia entre reproductores.

FASE 1: Configuración del Entorno y Estructura Base

1. Inicialización del Proyecto Frontend

Se recomienda **Vite + React + TypeScript** para garantizar un bundle ligero y tipado estricto en las marcas temporales.

```
npm create vite@latest syncreact -- --template react-ts
cd syncreact
npm install
npm install lucide-react react-youtube file-saver
npm install -D tailwindcss postcss autoprefixer
```

2. Definición del Esquema de Datos (`src/types/sync.ts`)

Crear las interfaces que definirán el contrato de datos del archivo `.sync`:

```
export interface SyncEvent {
  type: 'play' | 'pause' | 'seek';
  vrTimeSec: number; // Marca de tiempo del reaccionador (Master)
  vmTimeSec: number; // Marca de tiempo del video musical (Slave)
}

export interface SyncConfig {
  version: string;
  createdAt: string;
  vmUrl: string;
  events: SyncEvent[];
}
```

FASE 2: Módulo de Grabación de Video Reacción (/record)

Paso A: Captura de Cámara (MediaRecorder API)

- Crear un hook `useWebcam` con `navigator.mediaDevices.getUserMedia({ video: true, audio: true })`.
- Renderizar el feed en vivo en un componente `<video>` con la propiedad `muted` para acomodar encuadre e iluminación.
- Instanciar `MediaRecorder` configurando acumuladores en array de `Blob` (chunks).

Paso B: Embed e Integración de YouTube (VM)

- Integrar el reproductor IFrame de YouTube desactivando `autoplay`.
- Implementar parser para extraer el `videoId` mediante expresiones regulares estándar de URLs de YouTube.

Paso C: Registro de Marcas Temporales y Eventos

Al hacer clic en **GRABAR**:

- Iniciar la captura de la webcam: `mediaRecorder.start(1000)`.
- Establecer el tiempo de referencia cero (`startTime = Date.now()`).
- Escuchar el controlador `onStateChange` de la API IFrame de YouTube:
 - **PLAY** (`YT.PlayerState.PLAYING`): Registrar `{ type: 'play', vrTimeSec, vmTimeSec }`.
 - **PAUSE** (`YT.PlayerState.PAUSED`): Registrar `{ type: 'pause', vrTimeSec, vmTimeSec }`.
 - **SEEK**: Detectar saltos manuales en la línea de tiempo.

Paso D: Cierre y Exportación de Descargables

Al presionar detener:

- Detener el `MediaRecorder` y generar un archivo Blob unificado (`VR.mp4` / WebM).
- Exportar el JSON estructurado en un archivo descargable de extensión `.sync` mediante `FileSaver.js`.

FASE 3: Módulo de Publicación y Carga (`/upload`)

Formulario de Ingesta

Construir la pantalla con tres entradas principales:

- **Input 1:** URL original del Video Musical (VM).
- **Input 2:** URL de YouTube del video de la reacción (VR) subido previamente.
- **Input 3:** Lector de archivos para cargar el fichero `.sync` descargado.

Generación de la URL de Visitante (Elección de Arquitectura)

- **Opción 100% Client-Side (Sin Backend):** Serializar y comprimir el objeto JSON del `.sync` utilizando `lz-string` en Base64 e incrustarlo en los parámetros query de la URL URL.
Ejemplo: https://syncreact.app/watch?vm=ID_VM&vr=ID_VR&data=eJzLSM3...
- **Opción con Backend (Recomendada):** Guardar la configuración en base de datos (Supabase / Firebase / Node) y generar un identificador corto (slug).
Ejemplo: <https://syncreact.app/watch/v8aF9x2>

FASE 4: Reproductor Sincronizado para Visitantes (`/watch`)

Motor de Sincronización Master-Slave (`useSyncEngine`)

- El reproductor del Video Reacción (VR) actúa como **MASTER**.
- El reproductor del Video Musical (VM) actúa como **SLAVE**.
- Un bucle periódico (`requestAnimationFrame` o intervalo de 100ms) consulta la posición del Master:
 1. Lee el tiempo actual del VR (`vrCurrentTime`).
 2. Busca en el vector de eventos el estado correspondiente para ese segundo.
 3. Calcula el tiempo objetivo del VM: `vmTargetTime = vrCurrentTime - eventOffset`.

Controlador de Desfase (Drift Control) y Buffering

- **Corrección de latencia:** Si `Math.abs(vmRealTime - vmTargetTime) > 0.35` segundos, ejecutar `vmPlayer.seekTo(vmTargetTime)` para re-alinear sin interrumpir el audio.
- **Manejo de Buffering:** Si el reproductor VR entra en estado de buffering, pausar inmediatamente el VM. Al reanudar VR, reanudar VM.

FASE 5: Pruebas, Refinamiento y Despliegue

Validación y CI/CD

- Probar la reacción ante múltiples pausas y saltos aleatorios de la línea de tiempo.
- Desplegar en entornos como **Vercel** o **Netlify** conectando el repositorio de GitHub para compilación continua.